

Crear el login g1_sofia con la contraseña UCA@2026. Como es una persona, le dejamos activas la política de contraseñas y la expiración.

```
-- Crear login de Sofía
USE master;
GO

IF EXISTS (SELECT 1 FROM sys.server_principals WHERE name = N'g1_sofia')
    DROP LOGIN g1_sofia;
GO

CREATE LOGIN g1_sofia
WITH PASSWORD = 'UCA@2026',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON;
GO
```

Crear su usuario en Northwind, amarrado a ese login.

```
-- Crear usuario en DB_Ventas
USE DB_Ventas;
GO

CREATE USER g1_sofia
FOR LOGIN g1_sofia;
GO
```

Crear el rol g1_rol_catalogo, darle SELECT sobre dbo.Products, dbo.Categories y dbo.Suppliers, y meter a Sofía en el rol.

```
-- Crear rol del catálogo
CREATE ROLE g1_rol_catalogo;
GO

-- Dar permisos de consulta
GRANT SELECT ON dbo.Products
TO g1_rol_catalogo;

GRANT SELECT ON dbo.Categories
TO g1_rol_catalogo;

GRANT SELECT ON dbo.Suppliers
TO g1_rol_catalogo;
GO

-- Agregar a Sofía al rol
ALTER ROLE g1_rol_catalogo
ADD MEMBER g1_sofia;
GO
```

Probar con EXECUTE AS USER que puede sacar los 5 productos más caros con su categoría, pero que dbo.Orders no lo puede ver.

```
--hacemos la consulta de los primeros 5 mas caros
SELECT TOP 5
    p.ProductName,
    p.UnitPrice,
    c.CategoryName
FROM dbo.Products AS p
INNER JOIN dbo.Categories AS c
    ON p.CategoryID = c.CategoryID
ORDER BY p.UnitPrice DESC;
GO

-- Comprobamos que no tenga acceso a los Orders
BEGIN TRY
    SELECT *
    FROM dbo.Orders;
END TRY
BEGIN CATCH --Ahora si no se obtiene el acceso (como debe ser)
    SELECT 'Sofia no tiene acceso a dbo.Orders' AS Resultado;
END CATCH;
GO

-- Regresamos al usuario original
REVERT;
GO
```

Ejercicio 2:

1. Crear el login g1_precios (contraseña UCA@2026, con política y expiración), su usuario en Northwind y el rol g1_rol_precios, con el usuario adentro.

```
-- Crear login de precios
USE master;
GO

IF EXISTS (SELECT 1 FROM sys.server_principals WHERE name = N'g1_precios')
    DROP LOGIN g1_precios;
GO

--creamos otro login para precios
CREATE LOGIN g1_precios
WITH PASSWORD = 'UCA@2026',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON;
GO
```

2. Darle al rol SELECT sobre dbo.Products y UPDATE únicamente en UnitPrice y Discontinued.

```

-- Dar SELECT sobre Products
GRANT SELECT ON dbo.Products
TO g1_rol_precios;
GO

-- Damos UPDATE solamente en dos columnas
GRANT UPDATE (UnitPrice, Discontinued)
ON dbo.Products
TO g1_rol_precios;
GO

```

3. Como g1_precios, y dentro de una transacción que termina en ROLLBACK para no dañar los datos, intentar tres cosas: subir 10 % el precio de los productos de la categoría 1, dejar en 0 el UnitsInStock del producto 1 y borrar el producto 1.

```

-- Probamos permisos de precios
EXECUTE AS USER = 'g1_precios';
GO

BEGIN TRANSACTION;
GO

-- Aumentar 10% los precios de categoría 1
UPDATE dbo.Products
SET UnitPrice = UnitPrice * 1.10
WHERE CategoryID = 1;
GO

select * from Products;

-- Intentar eliminar producto
BEGIN TRY
    DELETE FROM dbo.Products
    WHERE ProductID = 1;
END TRY
BEGIN CATCH
    SELECT 'No tiene permiso para eliminar productos' AS Resultado;
END CATCH;
GO

-- Deshacer los cambios
ROLLBACK;
GO

REVERT;
GO

```

1. Crear el login g1_app_bodega (contraseña UCA@2026, con política pero sin expiración, porque es una aplicación), su usuario y el rol g1_rol_bodega.

```

-- Crear login de bodega
USE master;
GO

IF EXISTS (SELECT 1 FROM sys.server_principals WHERE name = N'g1_app_bodega')
    DROP LOGIN g1_app_bodega;
GO

CREATE LOGIN g1_app_bodega
WITH PASSWORD = 'UCA@2026',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = OFF;
GO

```

2. Darle al rol SELECT en dbo.Products y dbo.Categories, e INSERT y UPDATE en dbo.Products.

```

-- Comprobar permisos de bodega
EXECUTE AS USER = 'g1_app_bodega';
GO

SELECT
    'dbo.Products' AS Objeto,
    HAS_PERMS_BY_NAME('dbo.Products', 'OBJECT', 'SELECT') AS [SELECT],
    HAS_PERMS_BY_NAME('dbo.Products', 'OBJECT', 'INSERT') AS [INSERT],
    HAS_PERMS_BY_NAME('dbo.Products', 'OBJECT', 'UPDATE') AS [UPDATE],
    HAS_PERMS_BY_NAME('dbo.Products', 'OBJECT', 'DELETE') AS [DELETE],
    HAS_PERMS_BY_NAME('dbo.Products', 'OBJECT', 'ALTER') AS [ALTER]

UNION ALL

SELECT
    'dbo.Categories',
    HAS_PERMS_BY_NAME('dbo.Categories', 'OBJECT', 'SELECT'),
    HAS_PERMS_BY_NAME('dbo.Categories', 'OBJECT', 'INSERT'),
    HAS_PERMS_BY_NAME('dbo.Categories', 'OBJECT', 'UPDATE'),
    HAS_PERMS_BY_NAME('dbo.Categories', 'OBJECT', 'DELETE'),
    HAS_PERMS_BY_NAME('dbo.Categories', 'OBJECT', 'ALTER')

UNION ALL

SELECT
    'dbo.Suppliers',
    HAS_PERMS_BY_NAME('dbo.Suppliers', 'OBJECT', 'SELECT'),
    HAS_PERMS_BY_NAME('dbo.Suppliers', 'OBJECT', 'INSERT'),
    HAS_PERMS_BY_NAME('dbo.Suppliers', 'OBJECT', 'UPDATE'),
    HAS_PERMS_BY_NAME('dbo.Suppliers', 'OBJECT', 'DELETE'),
    HAS_PERMS_BY_NAME('dbo.Suppliers', 'OBJECT', 'ALTER');
GO

REVERT;
GO

```

3. Blindar la tabla con DENY DELETE y DENY ALTER sobre dbo.Products.
4. Simular el error del otro administrador: agregar g1_app_bodega al rol fijo db_datawriter.
5. Como g1_app_bodega, armar con HAS_PERMS_BY_NAME la matriz de SELECT, INSERT, UPDATE, DELETE y ALTER para dbo.Products, dbo.Categories y dbo.Suppliers, y explicar cada valor.

```
|  
-- Quitar el permiso accidental  
ALTER ROLE db_datawriter --quitamos el acceso al rol quitando a datawriter  
DROP MEMBER gl_app_bodega; --que eso pasa quitando directamente a bdega  
GO
```